

Microservices Architecture

1. What are Microservices?

Microservices is an architectural style used to build applications as a **collection of small, independent services**.

Each service:

- Is **self-contained**
- Focuses on a **single business capability**
- Communicates via **APIs (HTTP/REST, messaging, etc.)**
- Can be **developed, deployed, and scaled independently**

Goal:

- Faster development cycles
- Better scalability
- Independent teams & deployments

Compared to Monolith:

- Monolith → tightly coupled
- Microservices → loosely coupled

2. Why Microservices?

- Independent deployments
- Technology flexibility (Java, Node, Python, etc.)
- Better fault isolation
- Scalable per service
- Team autonomy

3. Spring Boot for Microservices

Spring Boot is widely used for building microservices because:

- Rapid development (auto-configuration)
- Embedded servers (Tomcat, Netty)
- Minimal boilerplate code
- Easy integration with **Spring Cloud**
- Production-ready features (Actuator, metrics)

Key idea:
Focus on **business logic**, not infrastructure

4. Core Architecture Components

1. API Gateway

- Single entry point for all clients
- Handles:
 - Routing
 - Authentication
 - Rate limiting
 - Logging

Example:

- Spring Cloud Gateway

2. Service Discovery & Registration

Problem: Services have dynamic IPs

Solution: Use a registry

- Services register themselves
- Other services discover them dynamically

Tools:

- Netflix Eureka
- Consul
- Zookeeper

3. Configuration Server

- Centralized configuration management
- Avoid hardcoding configs

Example:

- Spring Cloud Config Server

4. Database per Service

- Each microservice has its **own database**
- No shared DB across different microservices

Ensures:

- Loose coupling
- Independent scaling

5. Identity Provider (Security)

Centralized authentication & authorization

Examples:

- Keycloak
- Auth2
- Okta
- Spring Security + JWT

5. Communication Between Services

1. Synchronous

- REST APIs (HTTP)

Use when immediate response is required

2. Asynchronous

Event-driven communication

- Message brokers:
 - Apache Kafka
 - RabbitMQ

Benefits:

- Loose coupling
- Better scalability
- Fault tolerance

6. Resilience & Fault Tolerance

Circuit Breaker Pattern

- Prevents cascading failures
- Stops calling failing service

Example:

- Resilience4j

Features:

- Retry
- Timeout
- Bulkhead
- Fallback methods

7. Important Microservices Patterns

- API Gateway Pattern
- Service Discovery Pattern
- Circuit Breaker Pattern
- Saga Pattern (Distributed transactions)
- Event-Driven Architecture
- Database per Service

8. Observability

Essential for real-world applications

1. Logging

- Centralized logging
- ELK Stack (Elasticsearch, Logstash, Kibana)

2. Monitoring

- Metrics tracking
- Tools:
 - o Prometheus
 - o Grafana
 - o Spring Boot Actuator

3. Distributed Tracing

- Track request across services

Tools:

- Zipkin
- Jaeger

9. Containerization & Deployment

- Docker → containerization
- Kubernetes → orchestration

Benefits:

- Scalability
- High availability
- Auto-scaling

10. Development Steps

1. Define business service
2. Create Spring Boot project
3. Implement business logic
4. Add database layer
5. Enable inter-service communication
6. Add security
7. Test (unit + integration)
8. Containerize (Docker)
9. Deploy (Kubernetes / Cloud)
10. Monitor & log

11. Best Practices

- Design services around **business capabilities**
- Keep services **small and focused**
- Prefer **asynchronous communication** , if possible
- Avoid shared databases
- Implement **fault tolerance (Resilience4j)**
- Use **API Gateway**
- Use Centralize configuration
- Secure APIs (JWT / OAuth2)
- Automate deployment (CI/CD)

12. Challenges

- Distributed system complexity
- Debugging is harder
- Data consistency issues
- Network latency
- Deployment complexity
- Requires DevOps maturity

13. Real-World Example (ECommerce)

Architecture Highlights

- Each service has its **own database**
- Communication:
 - Sync → REST APIs
 - Async → Kafka (notifications, events)
- API Gateway → single entry point
- Service Discovery → dynamic communication
- JWT-based security across services